Guia de estudos rápido — Totem da Sorveteria

Architecture Challenge · Grupo 3 · UNIFOR — leia a seção da sua parte (≈ 15 min) e as perguntas do final (≈ 10 min). Caminhos de código relativos à raiz do projeto; `.../` abrevia `src/main/java/br/unifor/sorveteria/<serviço>/`.

Integrante	Leia primeiro
1 — Introdução	§0, §2, perguntas 1 e 2
2 — Arquitetura	§2, §4, §5, perguntas 1, 3, 4, 5, 6
3 — Padrões	§1, §3, perguntas 7, 8, 9, 10
4 — Demonstração	§5, `Roteiro_Demonstracao.md`, perguntas 5 e 11
5 — Testes	§6, perguntas 11 e 12

0. O sistema em 30 segundos

Um totem de autoatendimento: o cliente escolhe a **linha** (Tradicional ou Zero Açúcar), o **sabor**, até **3 coberturas** e a **quantidade**; usa o **carrinho** com **cupom** e **desfazer**; e **paga** com cartão ou Pix. Na atividade anterior era um programa de console em Java com 5 padrões. Agora são **4 microsserviços Spring Boot** + **front-end React**, com **9 padrões** e **85 testes**.

1. Os nove Design Patterns (explicação simples)

#	Padrão	Categoria	Em uma frase (analogia)	Onde está no código
1	**Abstract Factory**	Criacional	Uma "linha de produção" por linha de sorvete: a fábrica Zero só faz sorvete Zero **e** casquinha Zero, então nunca se misturam.	`catalogo-service/.../domain/abstractfactory/` — `FabricaLinha`, `FabricaZero`, `FabricaTradicional`, `FabricasPorLinha`
2	**Builder**	Criacional	Um "montador" que recebe as peças em qualquer ordem e só entrega o sorvete se estiver válido (sabor da linha, ≤ 3 coberturas, sem repetir).	`catalogo-service/.../domain/builder/SorveteBuilder.java` (`construir`, `validar`)
3	**Factory Method**	Criacional	O roteiro de cobrança é sempre o mesmo; cada tipo de processador (cartão/Pix) decide **qual** gateway criar no método `criarGateway()`.	`pagamento-service/.../domain/factorymethod/` — `ProcessadorPagamento`, `ProcessadorCartao`, `ProcessadorPix`

#	Padrão	Categoria	Em uma frase (analogia)	Onde está no código
4	**Decorator**	Estrutural	Cada cobertura "embrulha" o sorvete e soma descrição, preço e item de estoque — dá para empilhar: granulado sobre calda sobre sorvete.	`catalogo-service/.../domain/decorator/` — `CoberturaDecorator`, `CaldaChocolate`, `Granulado`, `Chantilly`, `CastanhaCaramelizada`
5	**Adapter**	Estrutural	Um "tradutor de tomada": o SDK da operadora fala centavos e códigos ("51"); o adaptador traduz para a nossa interface `GatewayPagamento`.	`pagamento-service/.../domain/adapter/CartaoGatewayAdapter.java` + `adapter/legado/LegacyCardProcessor.java`
6	**Facade**	Estrutural	Um "balcão único": o controller só fala com a `PedidoFacade`, que coordena clientes HTTP, comandos, histórico, banco e checkout.	`pedido-service/.../facade/PedidoFacade.java`
7	**Command**	Comportamental	Cada ação do carrinho vira um objeto que sabe **executar** e **desfazer**; os objetos ficam numa pilha.	`pedido-service/.../command/` — `Comando`, `ComandoPedido`, `AdicionarItemCommand`, `RemoverItemCommand`, `AplicarCupomCommand`, `HistoricoComandos`
8	**Memento**	Comportamental	Uma "foto" do pedido tirada antes de cada ação; só o próprio `Pedido` consegue ler a foto.	`pedido-service/.../domain/SnapshotPedido.java` + `Pedido.java` (`criarSnapshot`, `restaurar`)
9	**Mediator**	Comportamental	Um "controlador de tráfego" do checkout: estoque, pagamento e pedido não se falam; avisam o mediador, que decide o próximo passo.	`pedido-service/.../checkout/TotemCheckoutMediator.java` + `Participante*.java`

Herdados do projeto anterior: 1, 4, 7, 8, 9. Novos: 2, 3, 5, 6. **Não contamos** como padrão nosso: injeção de dependência, `JpaRepository`, proxies do Spring. Atalho: `grep -rn "PADRÃO:" */src/main` lista todos os arquivos dos padrões.

2. Os quatro microsserviços

Serviço	Porta	O que faz	Dono de (banco H2 próprio)
`catalogo-service`	8081	Cardápio, preços e montagem do sorvete (calcula o preço oficial)	`itens_catalogo`
`pedido-service`	8084	Carrinho, cupons, desfazer, status, checkout	`pedidos`, `itens_pedido`, `cupons`
`estoque-service`	8082	Saldo por SKU, reservas tudo-ou-nada, confirmação e liberação	`itens_estoque`, `reservas`

Serviço	Porta	O que faz	Dono de (banco H2 próprio)
`pagamento-service`	8083	Cartão e Pix simulados; nunca cobra duas vezes o mesmo pedido	`pagamentos` (só os 4 últimos dígitos do cartão)

- **Por que essa divisão?** Cada serviço é dono de dados que mudam por motivos diferentes (cardápio × vendas × saldo × cobranças).
- **Organização interna (Service Layer):** `api/` (Controller + tratamento de erros) → `service/` ou `facade/` → `domain/` (regras e padrões) → `repository/` (Spring Data JPA).
- **Por que não MVC?** Não há telas no back-end; a "view" é o React, em outro processo.
- **Front-end:** React + Vite (`frontend/`); o proxy do Vite encaminha `/api/catalogo` → 8081, `/api/estoque` → 8082, `/api/pagamentos` → 8083, `/api/pedidos` e `/api/cupons` → 8084.

3. SOLID e GRASP no projeto

Princípio	Exemplo concreto
S — Responsabilidade única	`SorveteBuilder` só monta; `PedidoController` só traduz HTTP; `ApiExceptionHandler` só converte erros
O — Aberto/Fechado	Nova linha = nova `FabricaLinha` com `@Component`; `FabricasPorLinha` não muda. Novo meio de pagamento = nova subclasse de `ProcessadorPagamento`
L — Liskov	Qualquer `CoberturaDecorator` substitui um `Sorvete`; `CartaoGatewayAdapter` e `PixGatewaySimulado` respeitam o mesmo contrato
I — Segregação de interfaces	Um cliente pequeno por serviço: `CatalogoClient`, `EstoqueClient`, `PagamentoClient`
D — Inversão de dependência	As fábricas dependem da interface `CatalogoItens`, não do JPA; a `PedidoFacade` depende de interfaces de cliente
Information Expert	`Pedido.getTotal()`, `ItemPedido.getSubtotal()`, `Cupom.aplicavelEm()`, `Reserva.liberar()`
Creator	`FabricaZero` cria `SorveteZero`; `Pedido` cria `SnapshotPedido`; `ItemPedido.novo(...)`
Controller	Controllers REST só delegam; `TotemCheckoutMediator` controla o caso de uso "finalizar"
Baixo acoplamento	Serviços se conhecem só por JSON (`ContratosRemotos.java`); participantes do checkout não se conhecem
Alta coesão	Um assunto por serviço; um pacote por padrão
Polimorfismo	`TipoDesconto.PERCENTUAL / VALOR_FIXO.calcular(...)`; `criarGateway()` — sem `if/switch` por tipo

Detalhes: `docs/04-solid-grasp.md`.

4. Comunicação entre os serviços

- **REST síncrono** com `RestClient` do Spring; um cliente por serviço (`pedido-service/.../client/*HttpClient.java`).
- **Contratos explícitos:** o JSON enviado/recebido está em `ContratosRemotos.java`; campos extras são ignorados.
- **Timeouts:** 2 s para conectar, 5 s para ler (`config/ClientesHttpConfig.java`).
- **Erros padronizados (RFC 7807)** com campo `codigo`: 400 dados inválidos · 402 pagamento recusado · 404 não encontrado · 409 estoque insuficiente / pedido não editável · 422 regra de negócio · 502 pagamento em verificação · 503 serviço indisponível.
- **Quem chama quem:** só o pedido-service chama os outros (catálogo para o preço; estoque para disponibilidade e reservas; pagamento para cobrar e reconciliar).

5. Checkout: aprovado, recusado e incerto

O `TotemCheckoutMediator` executa uma **saga**: (1) reservar estoque → (2) cobrar → (3) concluir.

Resultado da cobrança	Como o sistema sabe	O que acontece	HTTP
Aprovado	resposta `status: APROVADO`	confirma a reserva; pedido `PAGO`	200
Recusado	resposta `status: RECUSADO`	**libera a reserva** (compensação); pedido `PAGAMENTO_RECUSADO`, ainda editável	402
Comprovadamente não cobrado	dados inválidos, conexão recusada, operadora indisponível informada pelo serviço	libera a reserva; pedido volta para `ABERTO`	422 / 503
Incerto	timeout de leitura, erro 5xx desconhecido, resposta ilegível	**mantém a reserva**, bloqueia nova cobrança e cancelamento; pedido `PAGAMENTO_EM_VERIFICACAO`	502

- **Reconciliação:** `POST /api/pedidos/{id}/reconciliacao` só conclui como `PAGO` se o pagamento-service mostrar um pagamento aprovado; sem aprovação, o pedido continua bloqueado (decisão manual — limitação declarada).
- **Idempotência:** o pagamento-service devolve o pagamento aprovado existente em vez de cobrar de novo, mesmo com duas chamadas simultâneas (trava por pedido em `PagamentoService.processar`).
- **Cartões de teste:** `4111 1111 1111 1111` aprova · `4000 0000 0000 0002` recusa (saldo insuficiente) · `4000 0000 0000 9999` operadora fora do ar · Pix aprova até R$ 1.000.

6. Testes — resultados reais

Execuções registradas em `ENTREGA_FINAL/evidencias/` (29/10/2026, macOS, JDK 25, Maven 3.9.15):

Serviço	Testes	Principais classes de teste
catálogo	16	`PadroesCriacionaisEEstruturaisTest`, `CatalogoApiIntegrationTest`
estoque	11	`EstoqueServiceTest`, `EstoqueApiIntegrationTest`

Serviço	Testes	Principais classes de teste
pagamento	13	`AdapterEFactoryMethodTest`, `PagamentoApiIntegrationTest`, `PagamentoConcorrenteTest`
pedido	45	`PedidoTest`, `CommandMementoTest`, `TotemCheckoutMediatorTest`, `ContratosHttpTest`, `PedidoApiIntegrationTest`
Total	**85, 0 falhas**	`mvn test` na raiz

Ponta a ponta (`scripts/smoke-test.sh`, 4 serviços no ar): **22 verificações ok, 0 falhas**. Ferramentas: JUnit 5, AssertJ, Mockito, Spring Boot Test (MockMvc, `@MockitoBean`), `MockRestServiceServer`. Front-end: sem testes automatizados (limitação); percorrido num Chrome real por script — capturas em `ENTREGA_FINAL/capturas/`.

7. Perguntas prováveis do professor

1. Isso não é um monólito dividido em quatro portas? Não. Cada serviço tem seu banco e nenhum lê a tabela de outro. O pedido só sabe o saldo perguntando à API do estoque; o estoque nem conhece preços. Se o estoque cair, o catálogo continua respondendo.

2. O que vocês evoluíram em relação ao trabalho anterior? Mantivemos domínio, regras e os 5 padrões; acrescentamos 4 padrões, 4 serviços com banco próprio, estoque com reservas, pagamento simulado, cupons com validade e mínimo, BigDecimal, tratamento de erros e 85 testes. O exemplo R$ 17,50 / R$ 27,90 do projeto original é verificado por teste.

3. Por que Service Layer e não MVC? Não há views no back-end; a view é o React. A Service Layer separa o HTTP (controllers) das regras (domínio) e permite testar as regras sem servidor (ADR-002).

4. Por que não mensageria? O cliente está parado no totem esperando a resposta. REST síncrono com saga e compensação resolve com menos infraestrutura (ADR-006).

5. E se o pagamento for aprovado mas a resposta se perder? O sistema não sabe se o dinheiro saiu, então **não** libera o estoque nem permite nova cobrança: o pedido vai para `PAGAMENTO_EM_VERIFICACAO`. A reconciliação consulta o pagamento-service e só conclui com aprovação comprovada. E mesmo que a cobrança fosse repetida, o pagamento-service devolveria a aprovação já existente.

6. Dois clientes compram o último sorvete de limão ao mesmo tempo? A reserva é transacional e `ItemEstoque` tem `@Version` (bloqueio otimista): a segunda transação falha e não deixa saldo negativo; o checkout desse pedido não é concluído.

7. Qual a diferença entre Abstract Factory e Factory Method? Abstract Factory: um **objeto** que cria uma **família** de produtos (sorvete + casquinha). Factory Method: um **método** sobrescrito por **subclasses** para criar **um** produto (`criarGateway()`).

8. Por que o Mediator mudou de lugar? No projeto antigo ele coordenava painéis de tela, que agora são do React. No back-end, o ponto em que várias peças reagem umas às outras é o checkout: o mediador evita que estoque, pagamento e pedido dependam diretamente uns dos outros.

9. O Command recebe o Pedido no `executar(Pedido)`. Ainda é Command? Sim: o comando encapsula a ação e o estado para desfazê-la (o Memento), e quem pede a ação não sabe como desfazê-la. Em REST o pedido é recarregado do banco a cada requisição, por isso o receptor é entregue na execução.

10. Onde o Open/Closed aparece de verdade? Para criar uma nova linha de sorvete basta uma nova classe `FabricaLinha` com `@Component`; `FabricasPorLinha` recebe todas no construtor, sem `if`. O mesmo vale para novos meios de pagamento.

11. Como vocês testaram a comunicação entre os serviços? `ContratosHttpTest` simula os outros serviços com `MockRestServiceServer` e verifica o JSON enviado e a interpretação de cada resposta (201, 409, 422, 500, 503, timeout, conexão recusada). Com tudo de pé, `smoke-test.sh` faz 22 verificações reais.

12. Quais são as limitações? Bancos H2 em memória; histórico de desfazer em memória; pedido em verificação sem aprovação exige decisão manual; trava contra cobrança dupla vale para uma instância; sem autenticação. Cada uma está documentada (ADRs e backlog B21–B29).